

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

جامعة القصارف
كلية علوم الحاسوب وتقانة المعلومات

اسم المقرر: نظم الوسائط المتعددة (2)
الدفعة (12) (تقانة المعلومات)

الفصل الدراسي التاسع
المحاضرة الثالثة

اعداد: د. ياسر بله عبدالسلام
yassir32@gmail.com

HUFFMAN CODING

خوارزمية HUFFMAN

تعتبر خوارزمية Huffman من أولى الخوارزميات الفعالة لضغط المعلومات (و ليس تشفيرها! مع أن عملية التشفير تتطلب القيام بعملية ضغط في البداية وذلك لخفض حجم البيانات ولحذف التشابه و التكرار مما يجعل عملية تتبع الأنماط pattern matching صعبة و بالتالي تصبح عملية فك تشفيرها صعبة)، وقد ظهرت هذه الخوارزمية أول مرة على يد العالم Huffman و ذلك بعد أعمال كثيرة لـ Shannon و Globe و غيرهم من رواد ضغط المعلومات.

إن تمثيل المحارف (وبشكل عام أي معلومة) النظامي في الحاسب يضع طولاً ثابتاً محددًا لكل محرف و هو 8 bits أو 1 byte لكل محرف في تمثيل الـ ASCII و هذا ما نعبر عنه بالـ "تمثيل ذي الطول الثابت" fixed length code، و تكمن فكرة خوارزمية هوفمان (و أية خوارزمية تعتمد الطرق الإحصائية statistical methods) في إعطاء المحارف ذات التواتر الأكبر أو التي احتمال ورودها أكبر تمثيلاً أقصر والمحارف ذات التواتر الأقل أو التي احتمال ورودها أقل تمثيلاً أطول، وأن لا يكون أي ترميز مقدمة لترميز آخر.

Huffman Coding Algorithm

- ✓ This technique was developed by David Huffman as part of a class assignment; the class was the first ever in the area of information theory and was taught by Robert Fano at MIT.
- ✓ The Codes generated using this technique are called Huffman Codes.
- ✓ These Codes are
 - ✓ Prefix codes
 - ✓ Optimum for a given model (set of probabilities)
- ✓ Based on two observations regarding optimum prefix codes:
 1. In an optimum code, symbols that occurs more frequently (have a higher probability of occurrence) will have short codewords than symbols that occur less frequently.
 2. In an optimum code, the two symbols that occur least frequently will have the same length.

Huffman Coding Algorithm

Example 3.2.1 Design of a Huffman Code

An alphabet $A = \{a_1, a_2, a_3, a_4, a_5\}$ with

$$P(a_1) = P(a_3) = 0.2$$

$$P(a_2) = 0.4$$

$$P(a_4) = P(a_5) = 0.1$$

$$\begin{aligned}\text{The entropy} &= -2 * 0.2 \log_2(0.2) - 0.4 \log_2(0.4) - 2 * 0.1 \log_2(0.1) \\ &= 2.122 \text{ bits/symbol}\end{aligned}$$

Table 3.1 The initial five-letter alphabet

Letter	Probability	Codeword
a_2	0.4	$c(a_2)$
a_1	0.2	$c(a_1)$
a_3	0.2	$c(a_3)$
a_4	0.1	$c(a_4)$
a_5	0.1	$c(a_5)$

The two symbols with the lowest probability are a_4 and a_5 .

$$c(a_4) = \alpha_1 * 0$$

$$c(a_5) = \alpha_1 * 1$$

α_1 is a binary string

Huffman Coding Algorithm

Define a new alphabet $A' = \{ a_1, a_2, a_3, a'_4 \}$
 where a'_4 is composed of a_4 and a_5 .
 $P(a'_4) = P(a_4) + P(a_5) = 0.2$

Table 3.2 The reduced four-letter alphabet

Letter	Probability	Codeword
a_2	0.4	$c(a_2)$
a_1	0.2	$c(a_1)$
a_3	0.2	$c(a_3)$
a'_4	0.2	α_1

In this alphabet A' ,
 a_3 and a'_4 are the two letters
 at the bottom of the sorted list.
 We assign their codewords as

$$c(a_3) = \alpha_2 * 0$$

$$c(a'_4) = \alpha_2 * 1$$

but $c(a'_4) = \alpha_1$.

Therefore, $\alpha_1 = \alpha_2 * 1$

Which mean that

$$c(a_4) = \alpha_1 * 0 = \alpha_2 * 10$$

$$c(a_5) = \alpha_1 * 1 = \alpha_2 * 11$$

Huffman Coding Algorithm

We again define a new alphabet $A'' = \{ a_1, a_2, a'_3 \}$
 where a'_3 is composed of a_3 and a'_4 .
 $P(a'_3) = P(a_3) + P(a'_4) = 0.4$

Table 3.3 The reduced three-letter alphabet

Letter	Probability	Codeword
a_2	0.4	$c(a_2)$
a'_3	0.4	α_2
a_1	0.2	$c(a_1)$

In this case, the least probable symbols are a'_3 and a_1 .

Therefore,

$$\begin{aligned} c(a'_3) &= \alpha_3 * 0 \\ c(a_1) &= \alpha_3 * 1 \end{aligned}$$

but $c(a'_3) = \alpha_2$.

Therefore, $\alpha_2 = \alpha_3 * 0$

Which mean that

$$c(a_3) = \alpha_2 * 0 = \alpha_3 * 00$$

$$c(a_4) = \alpha_2 * 10 = \alpha_3 * 010$$

$$c(a_5) = \alpha_2 * 11 = \alpha_3 * 011$$

Huffman Coding Algorithm

We again define a new alphabet $A''' = \{a''_3, a_2\}$
 where a''_3 is composed of a'_3 and a_1 .
 $P(a''_3) = P(a'_3) + P(a_1) = 0.6$

Table 3.4 The reduced two-letter alphabet

Letter	Probability	Codeword
a''_3	0.6	α_3
a_2	0.4	$c(a_2)$

We have only two letters,
 The codeword assignment is
 straightforward:

$$\begin{aligned} c(a''_3) &= 0 \\ c(a_2) &= 1 \end{aligned}$$

but $c(a''_3) = \alpha_3$.

Therefore, $\alpha_3 = 0$

Which mean that

$$c(a_1) = \alpha_3 * 1 = 01$$

$$c(a_3) = \alpha_3 * 00 = 000$$

$$c(a_4) = \alpha_3 * 010 = 0010$$

$$c(a_5) = \alpha_3 * 011 = 0011$$

Huffman Coding Algorithm

Table 3.5 Huffman code for the original five-letter alphabet

Letter	Probability	Codeword
a_2	0.4	1
a_1	0.2	01
a_3	0.2	000
a_4	0.1	0010
a_5	0.1	0011

The average length for this code is

$$l = .4*1 + .2*2 + .2*3 + .1*4 + .1*4 = 2.2 \text{ bits/symbol.}$$

A measure of the efficiency of this code is its

redundancy - the difference between the entropy and the average length.

In this case, the redundancy = $2.2 - 2.122 = 0.078$ bits/symbol.

Huffman Coding Algorithm

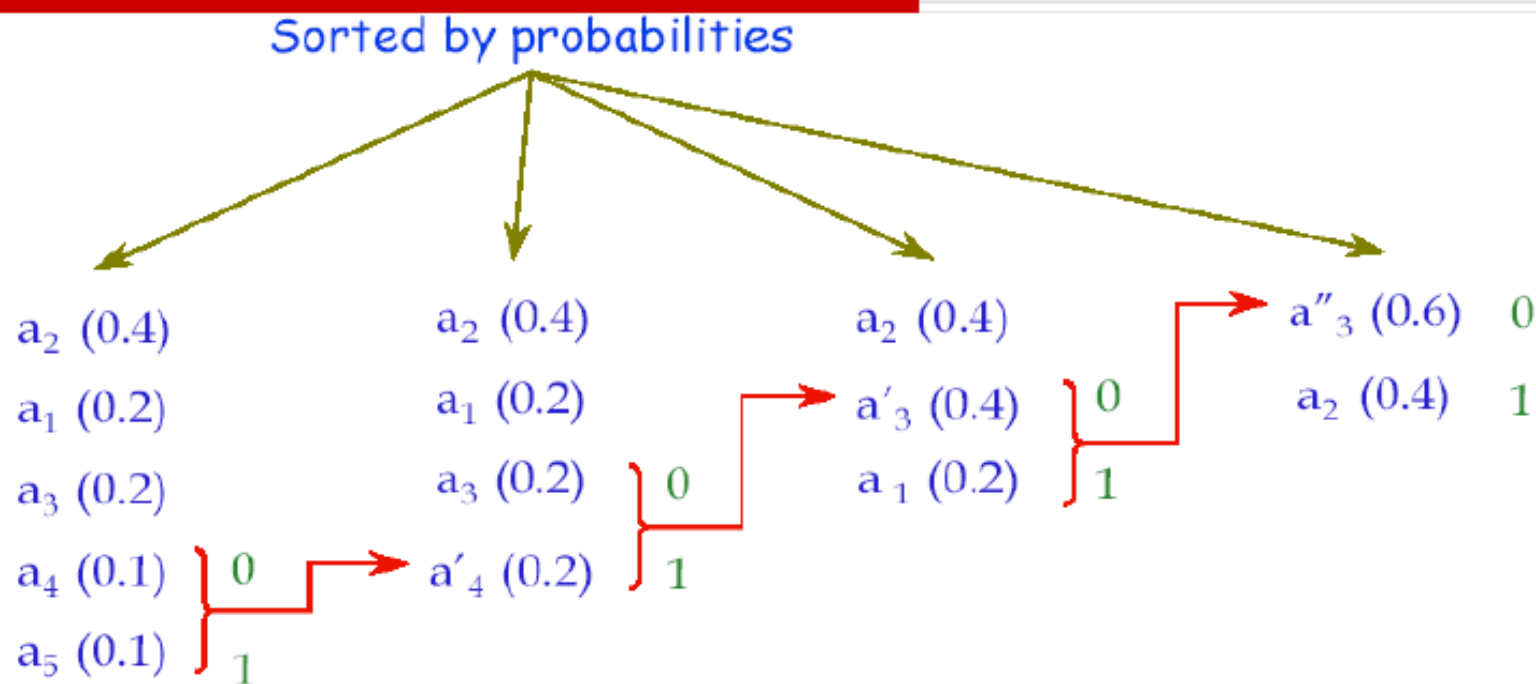


Figure 3.1 The Huffman encoding procedure.
The symbol probabilities are list in parentheses.

Huffman Coding Algorithm

We build the binary tree starting at the leaf nodes.

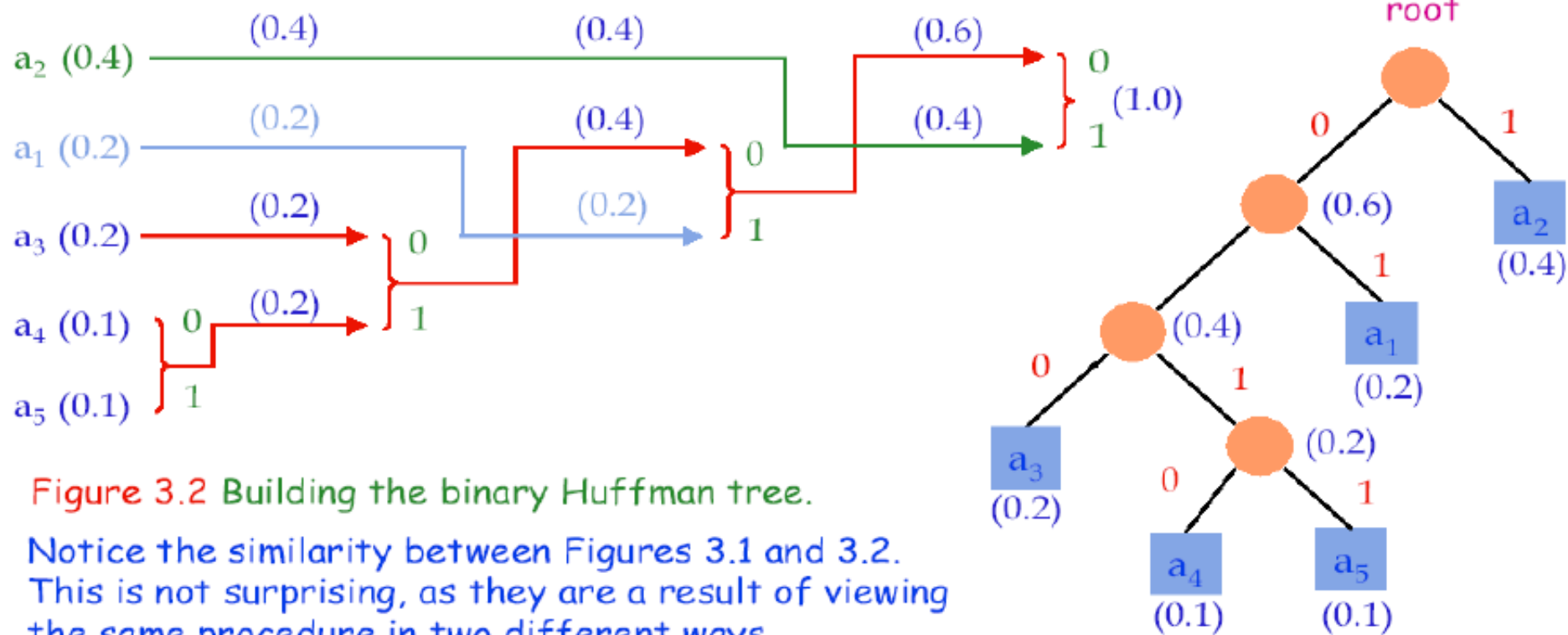


Figure 3.2 Building the binary Huffman tree.

Notice the similarity between Figures 3.1 and 3.2. This is not surprising, as they are a result of viewing the same procedure in two different ways.

شرح مبسط لكيفية ضغط الملفات بطريقة HUFFMAN: يمكن تلخيص خطوات إنشاء أكواد Huffman كما يلي:

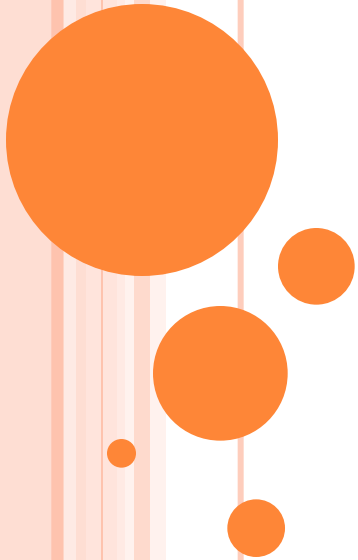
✓ إيجاد عدد مرات تكرار كل حرف في الملف النصي.

✓ يتم تكوين قائمة من العناصر كل عنصر يحتوي علي الرمز و عدد مرات تكراره وهذه العناصر ستكون الأوراق - Leafs - للشجرة الثنائية.

✓ اختر العنصرين من القائمة الذين لديهم أقل عدد مرات تكرار و اجمع أرقام التكرار لكل منهم لتحصل علي عنصر جديد يحتوي علي المجموع و يكون الابن -child- الأيمن لهذا العنصر الجديد هو العنصر الأقل تكرارا في القائمة و الابن الأيسر له هو العنصر الأقل الذي يليه , ثم احذف العنصران اللذان تم اختيارهما من القائمة و أضف العنصر الجديد في القائمة بترتيبه .

✓ يتم تكرار الخطوة رقم 3 لحين الحصول علي عنصر واحد في القائمة هذا العنصر سيكون ال root للشجرة الثنائية التي سيتم توليد الأكواد بواسطتها.

مثال يوضح كيفية الضغط وفك الضغط باستخدام خوارزمية
هوفمان عن طريق تطبيق الأشجار الثنائية:



BUILDING A TREE

SCAN THE ORIGINAL TEXT

- n Consider the following short text:

Eerie eyes seen near lake.

- n Count up the occurrences of all characters in the text



BUILDING A TREE

SCAN THE ORIGINAL TEXT

Eerie eyes seen near lake.

n What characters are present?

**E e r i space
y s n a r l k .**



BUILDING A TREE

SCAN THE ORIGINAL TEXT

Eerie eyes seen near lake.

n What is the frequency of each character in the text?

Char	Freq	Char	Freq	Char	Freq
E	1	y	1	k	1
e	8	s	2	.	1
r	2	n	2		
I	1	a	2		
space	4	l	1		



BUILDING A TREE

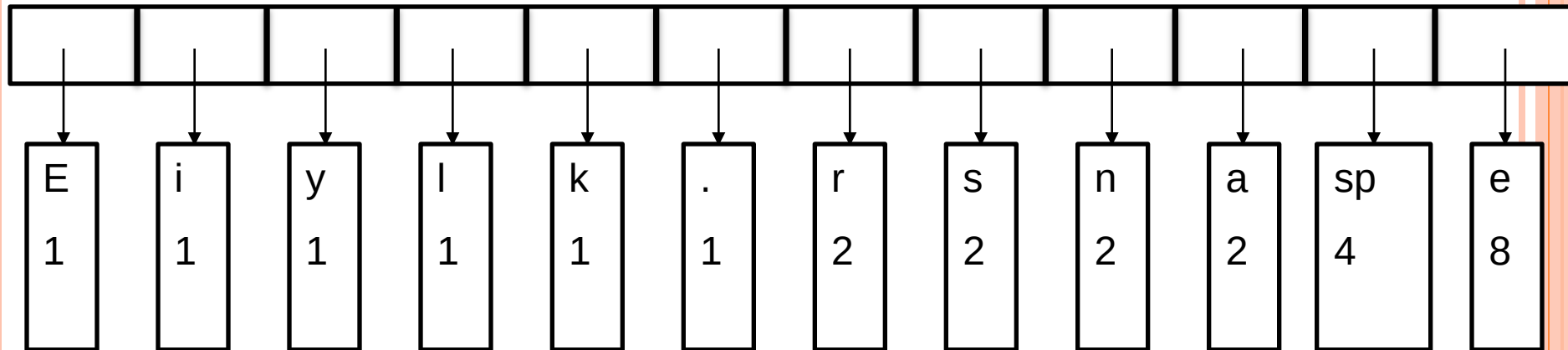
PRIORITIZE CHARACTERS

- n Create binary tree nodes with character and frequency of each character
- n Place nodes in a priority queue
 - The lower the occurrence, the higher the priority in the queue



BUILDING A TREE

n The queue after inserting all nodes



n Null Pointers are not shown



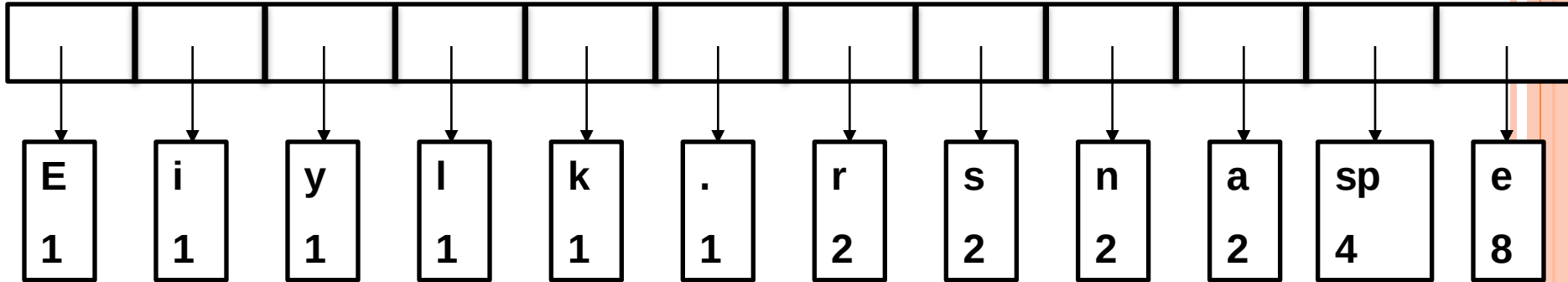
BUILDING A TREE

n While priority queue contains two or more nodes

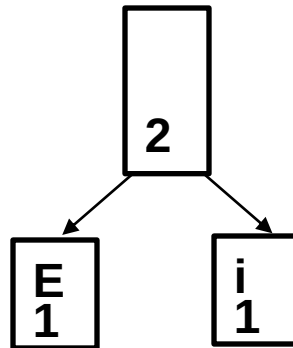
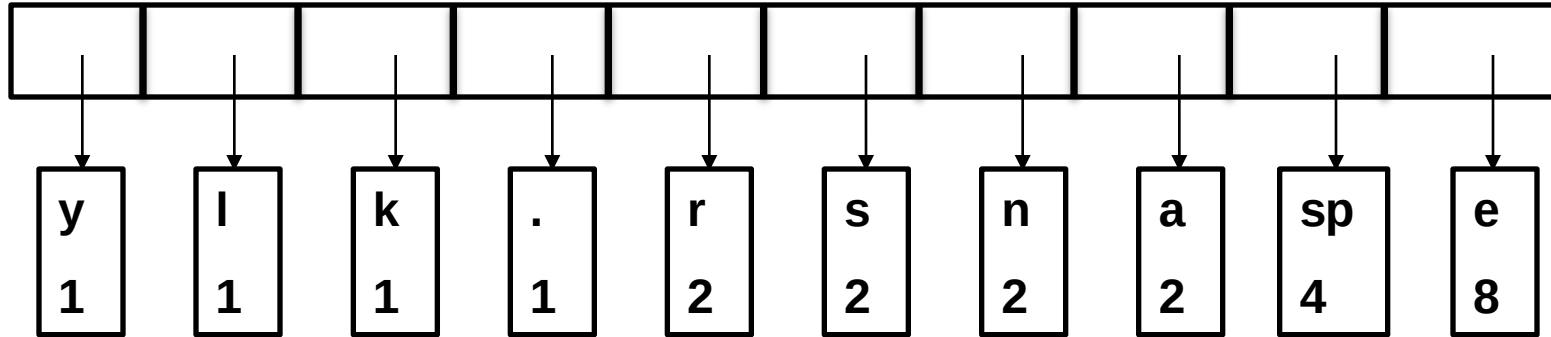
- Create new node
- Dequeue node and make it left subtree
- Dequeue next node and make it right subtree
- Frequency of new node equals sum of frequency of left and right children
- Enqueue new node back into queue



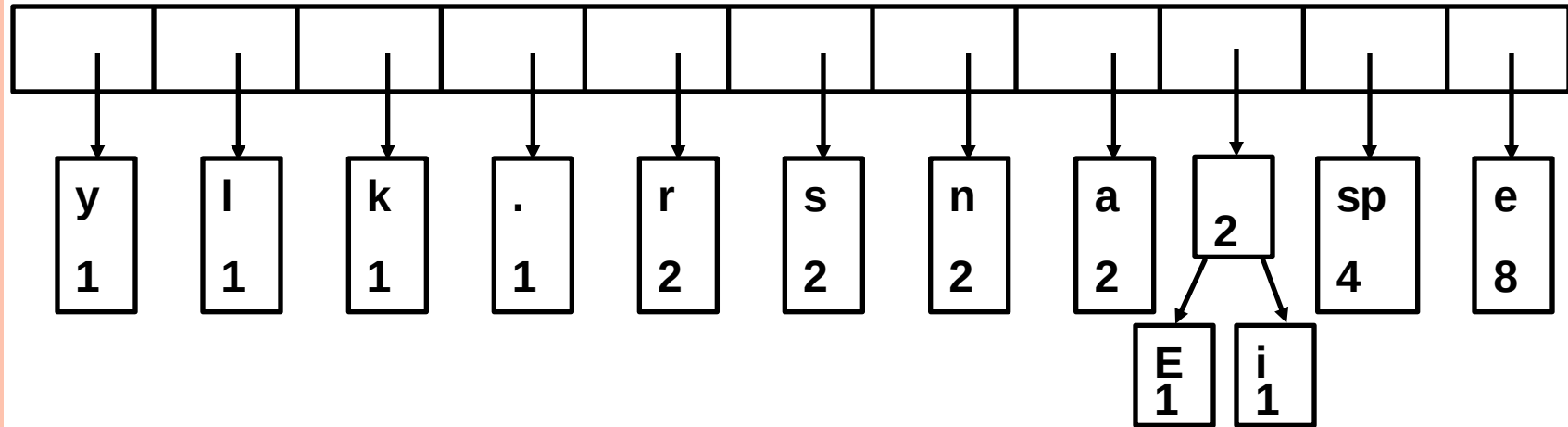
BUILDING A TREE



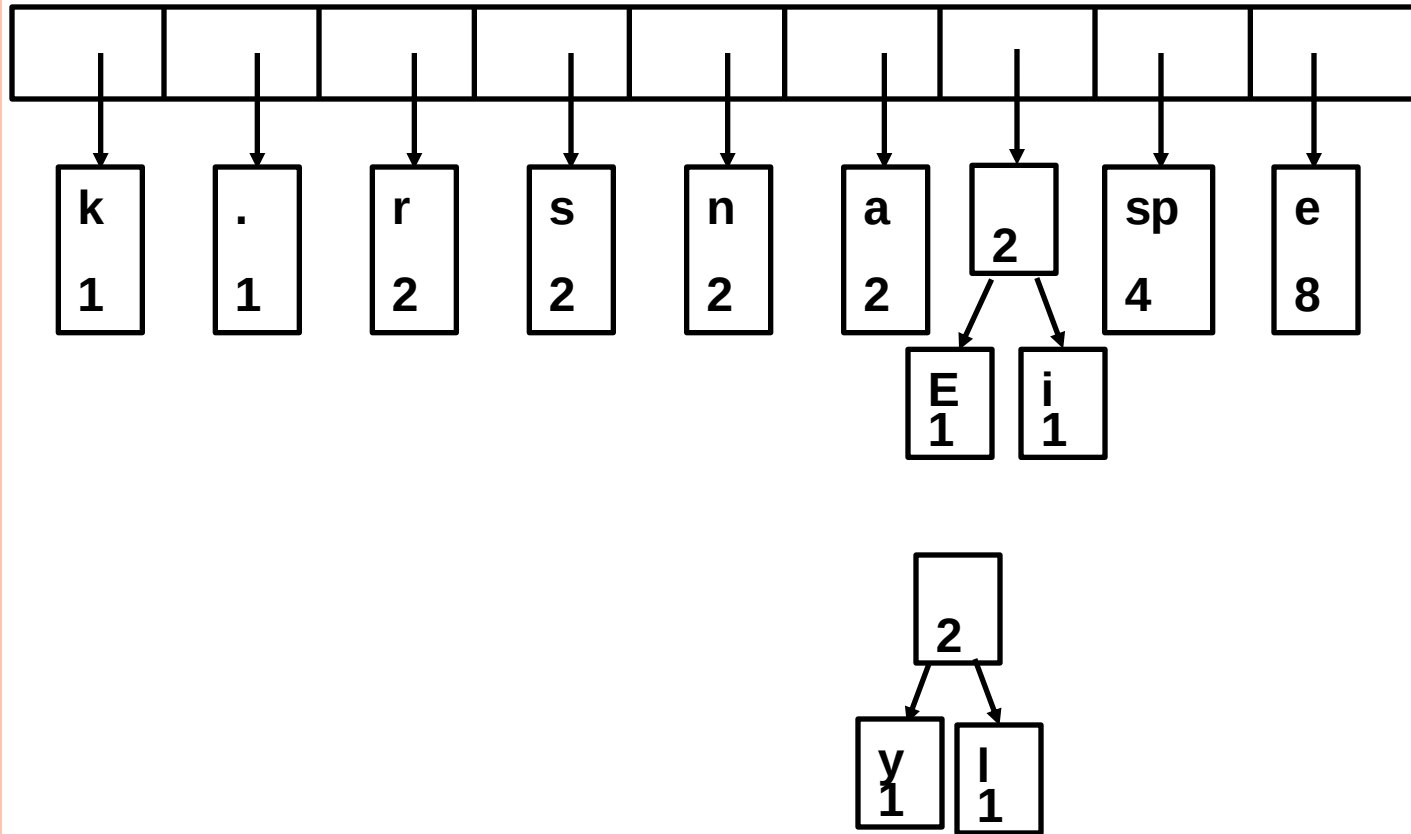
BUILDING A TREE



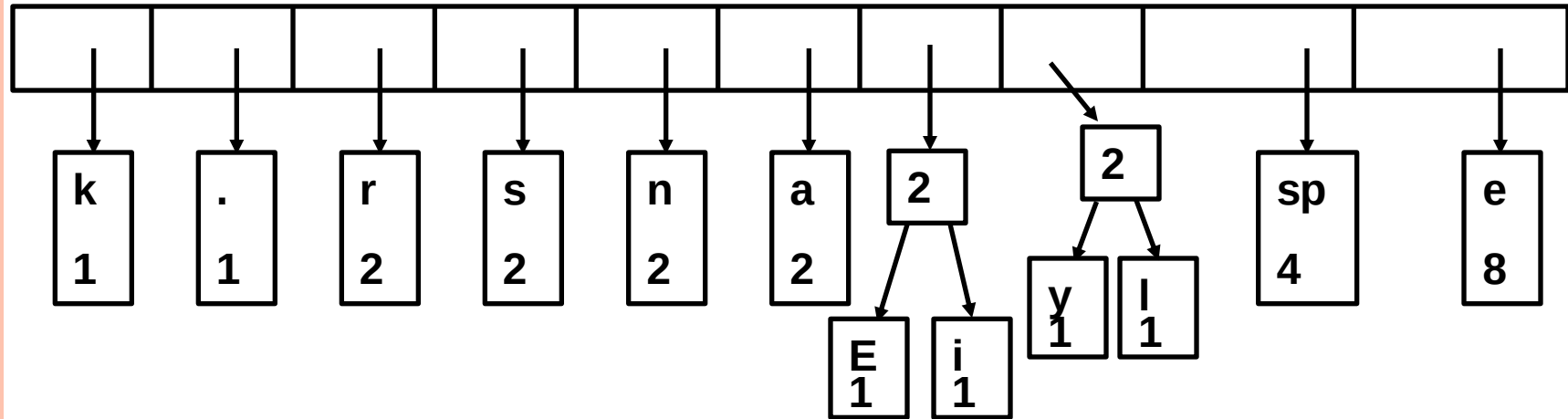
BUILDING A TREE



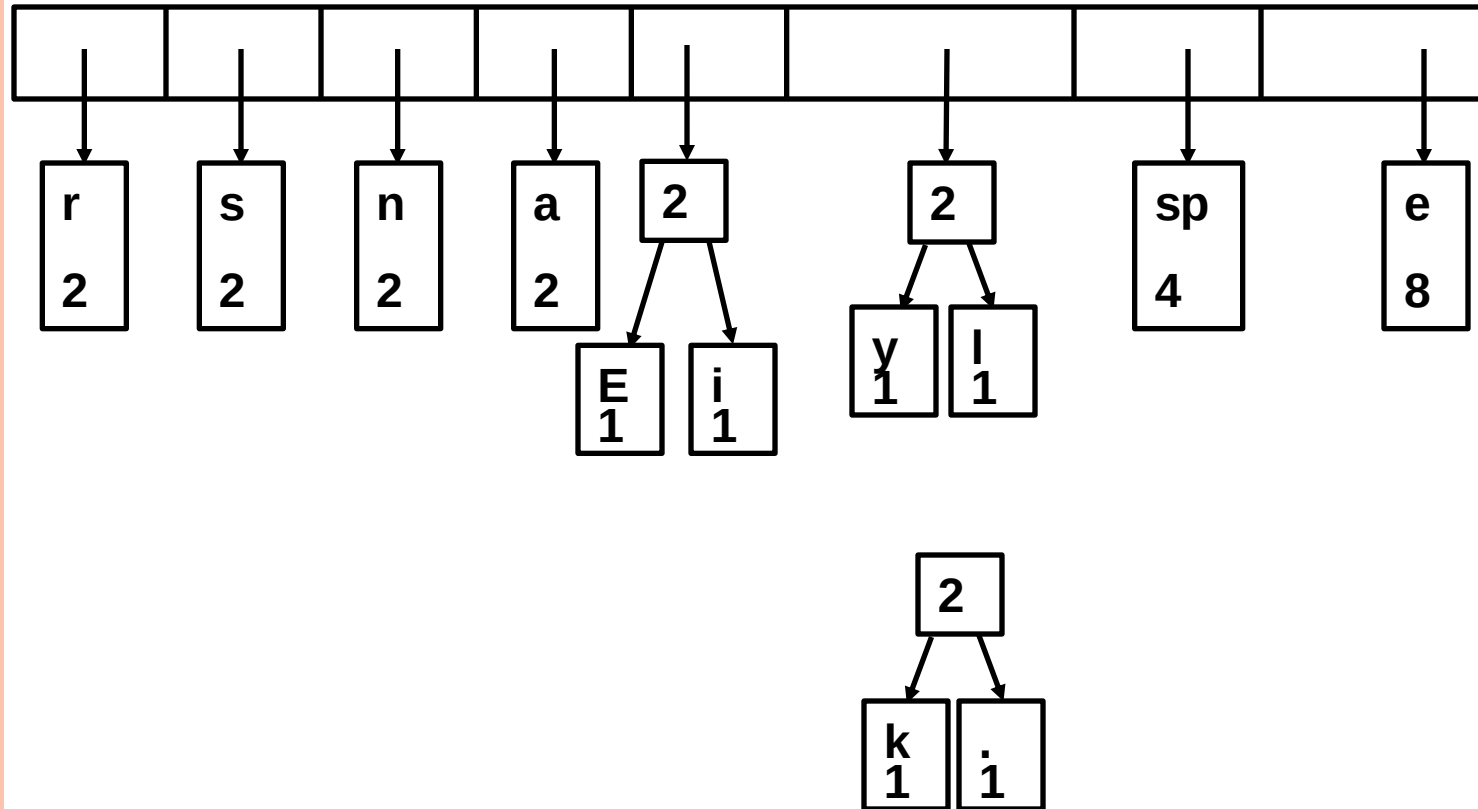
BUILDING A TREE



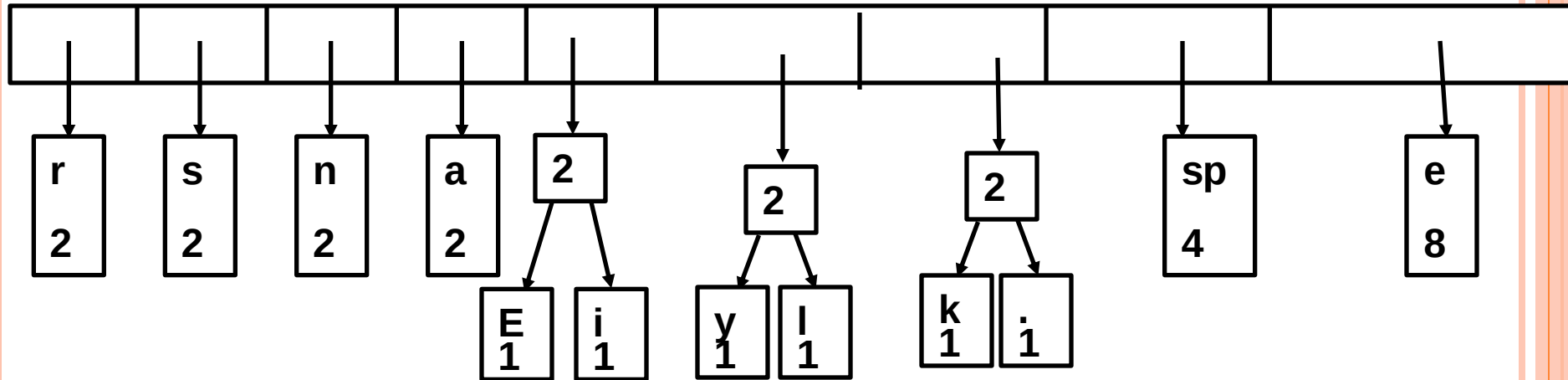
BUILDING A TREE



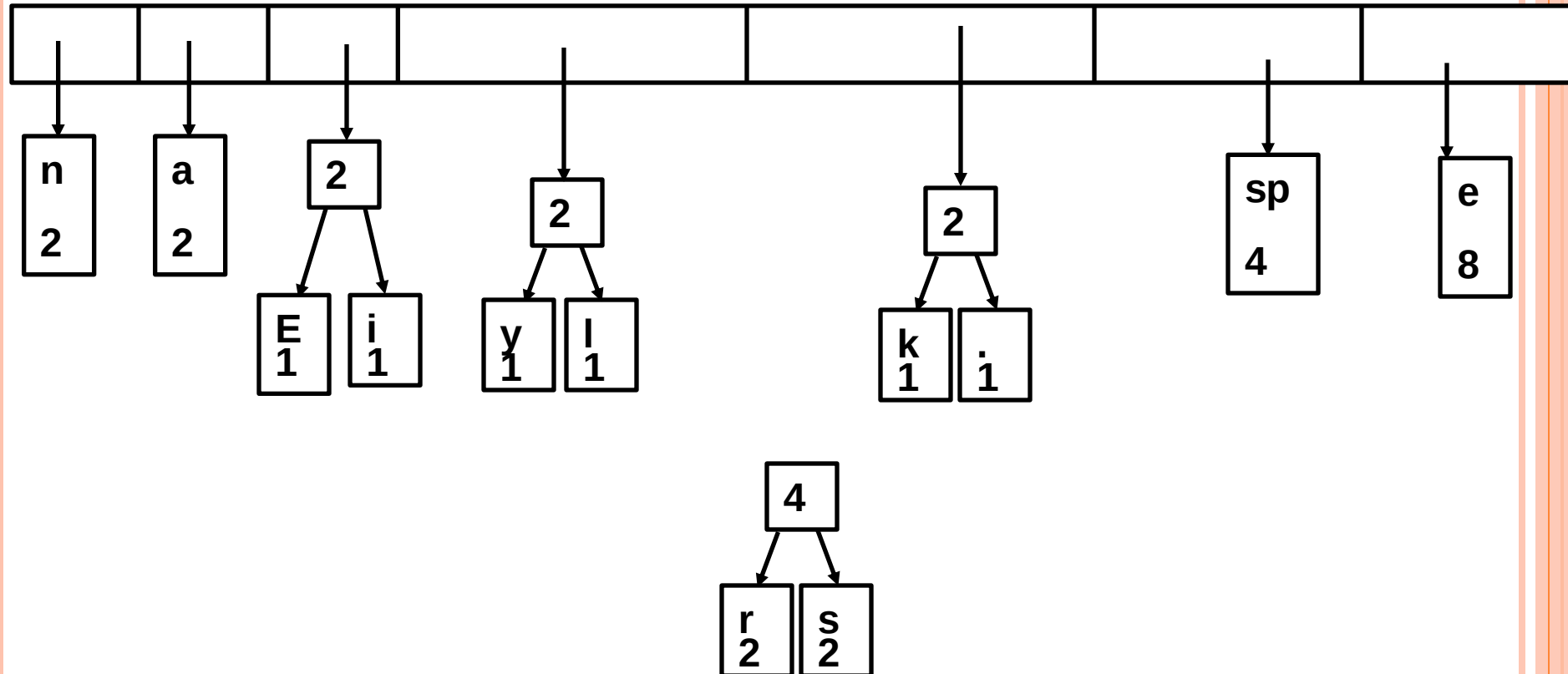
BUILDING A TREE



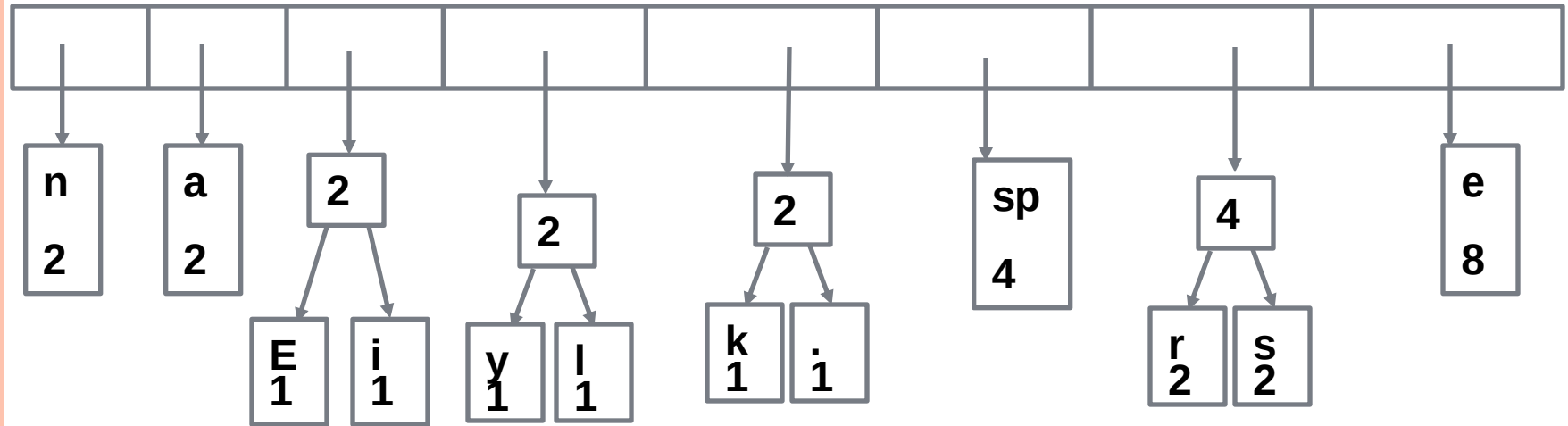
BUILDING A TREE



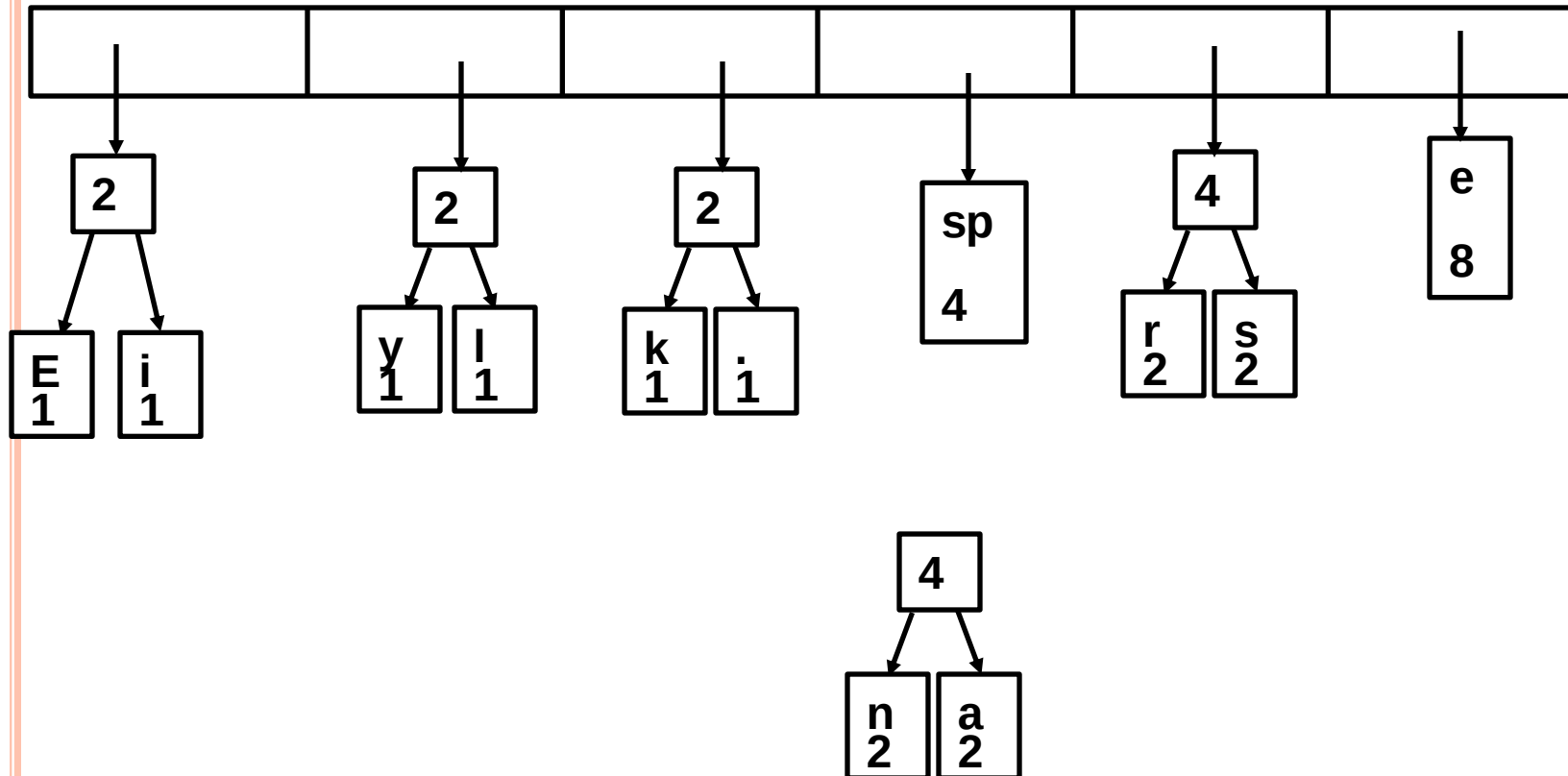
BUILDING A TREE



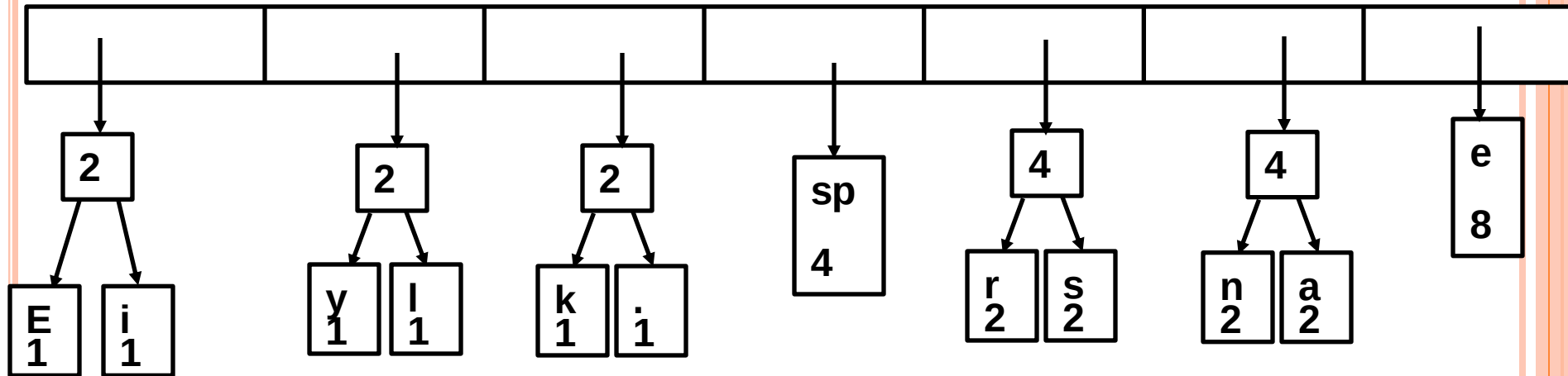
BUILDING A TREE



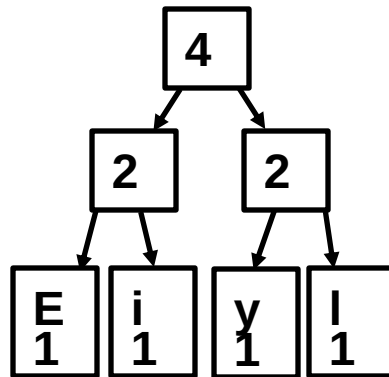
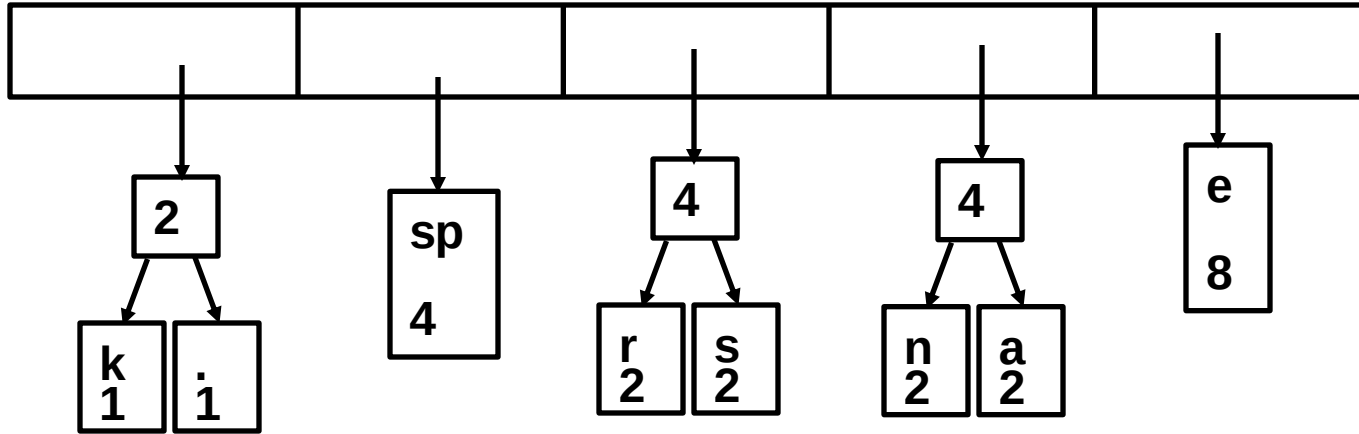
BUILDING A TREE



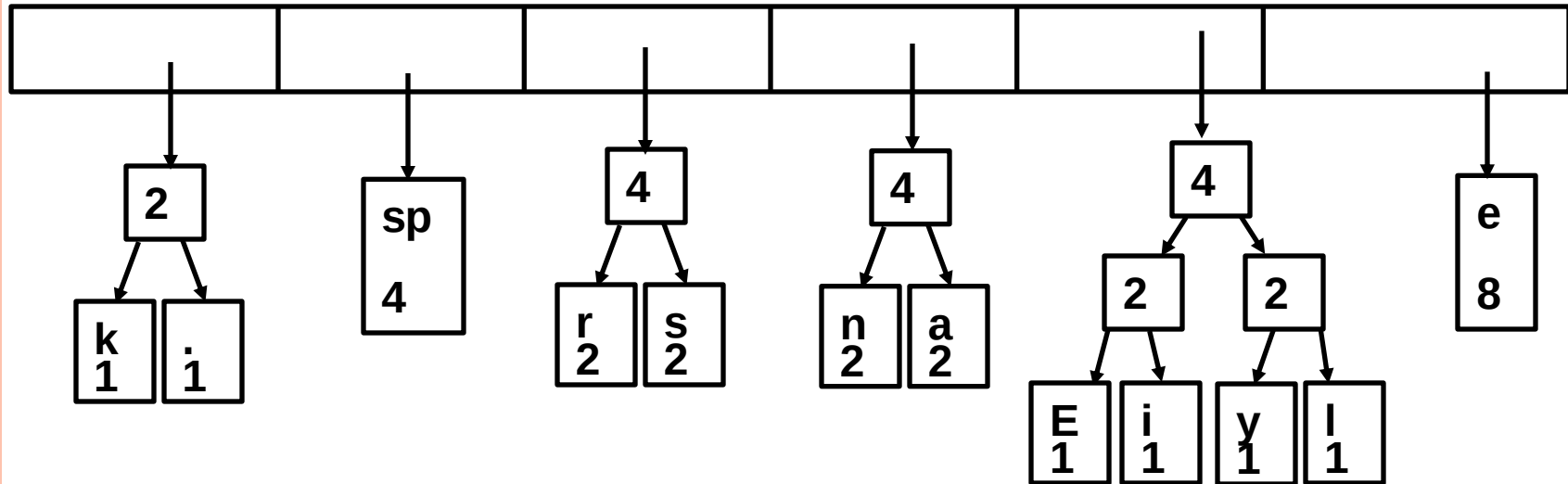
BUILDING A TREE



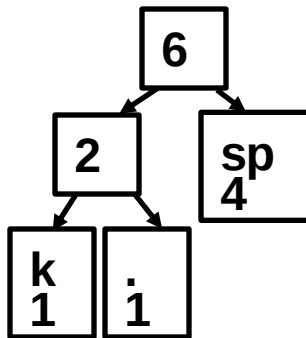
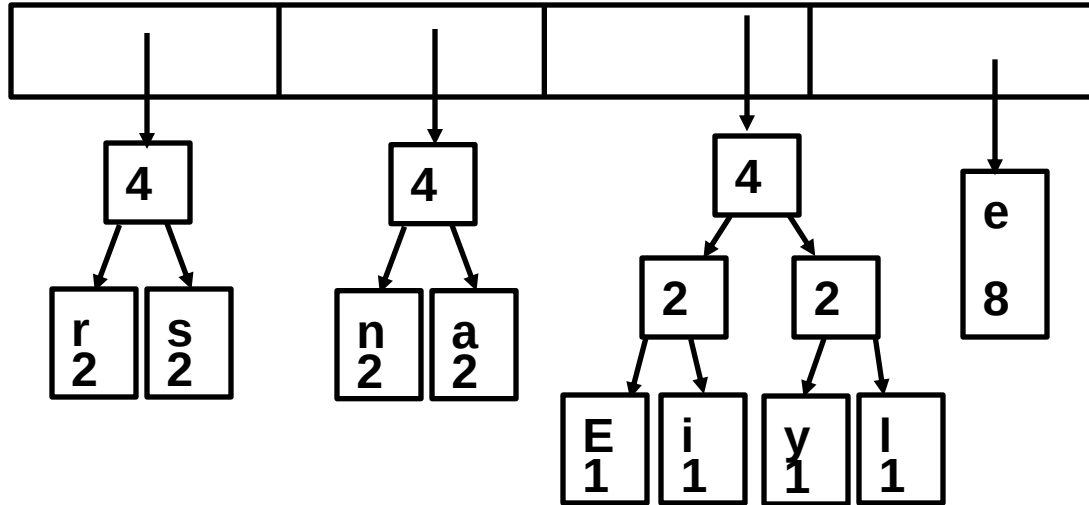
BUILDING A TREE



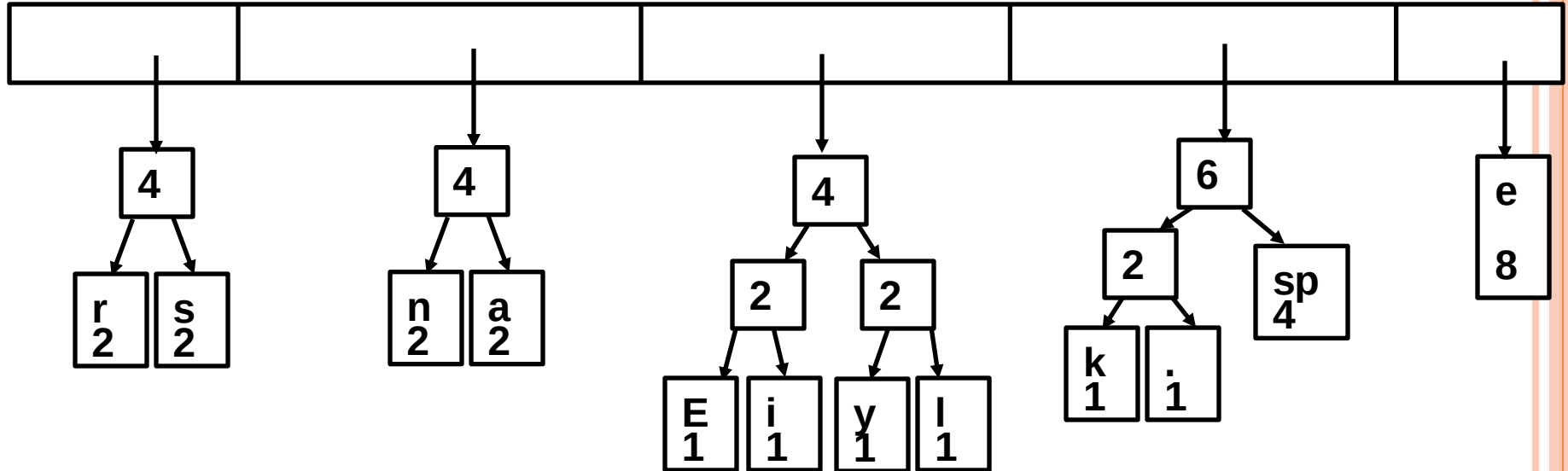
BUILDING A TREE



BUILDING A TREE



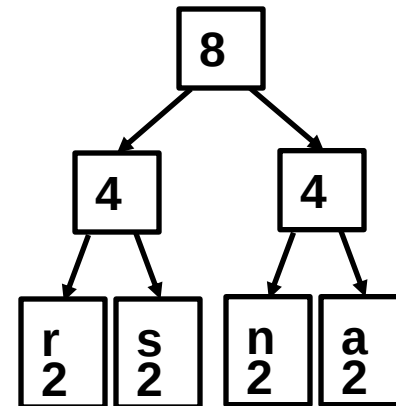
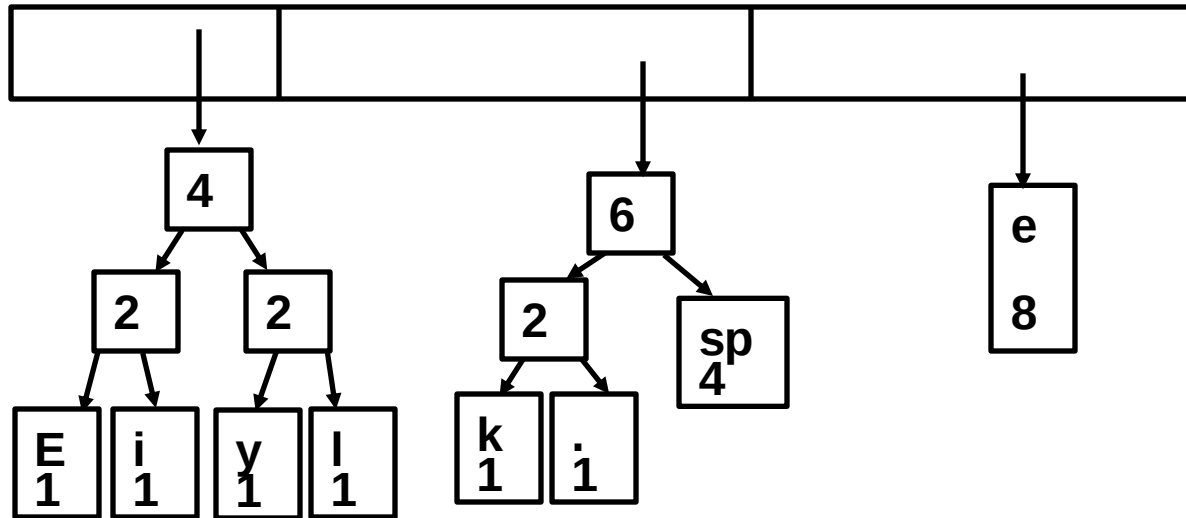
BUILDING A TREE



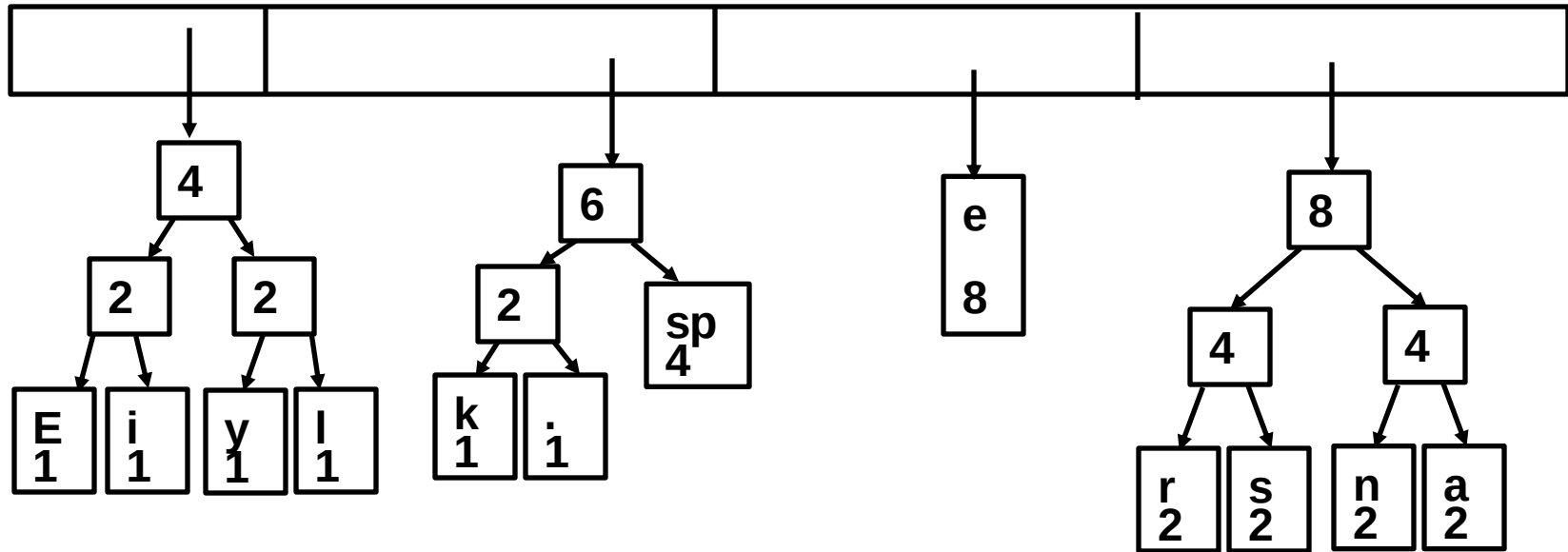
What is happening to the characters with a low number of occurrences?



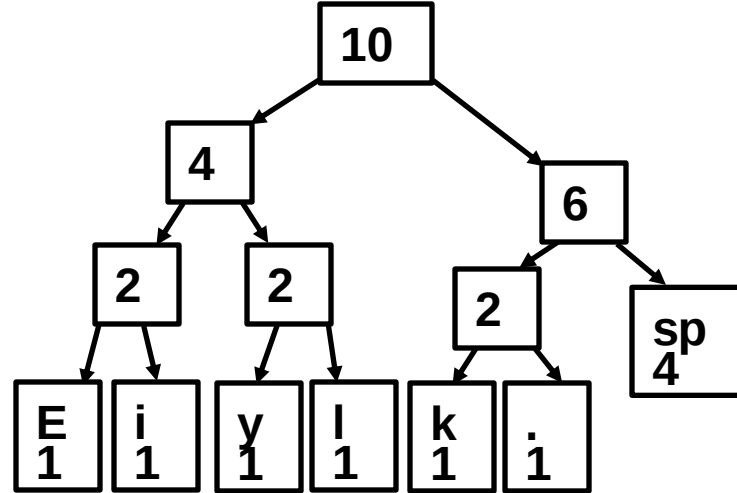
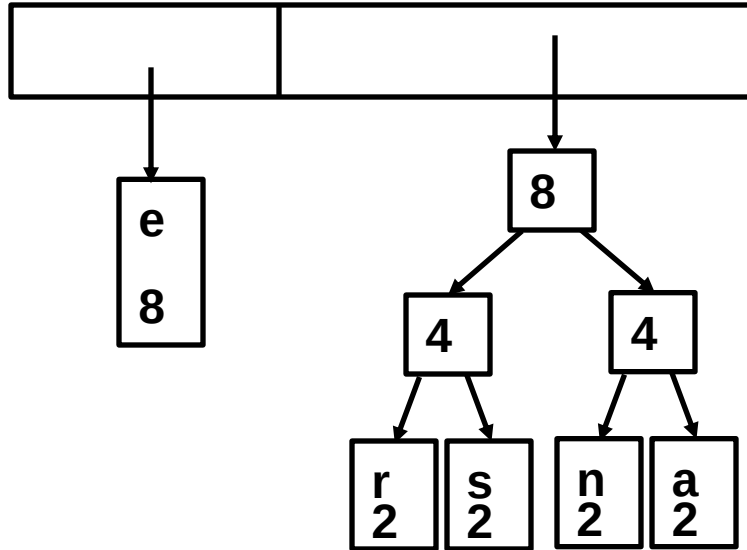
BUILDING A TREE



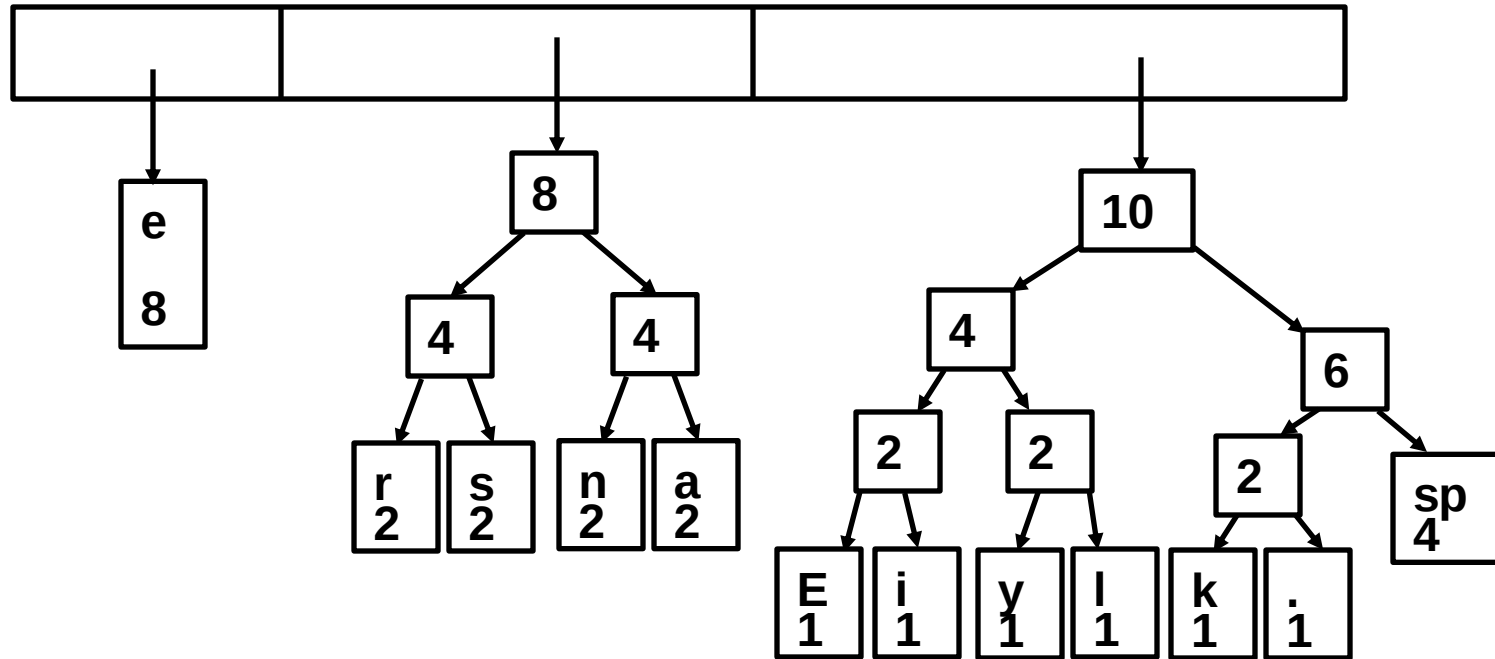
BUILDING A TREE



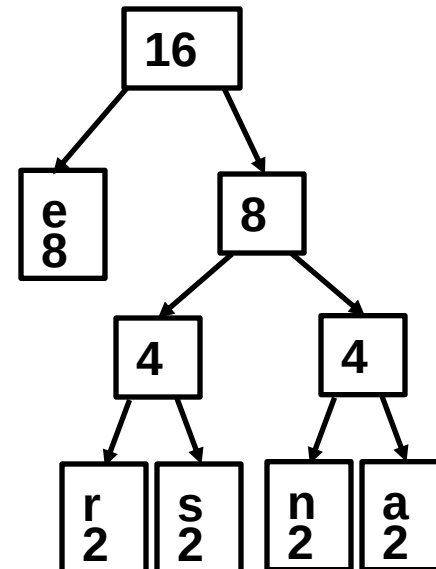
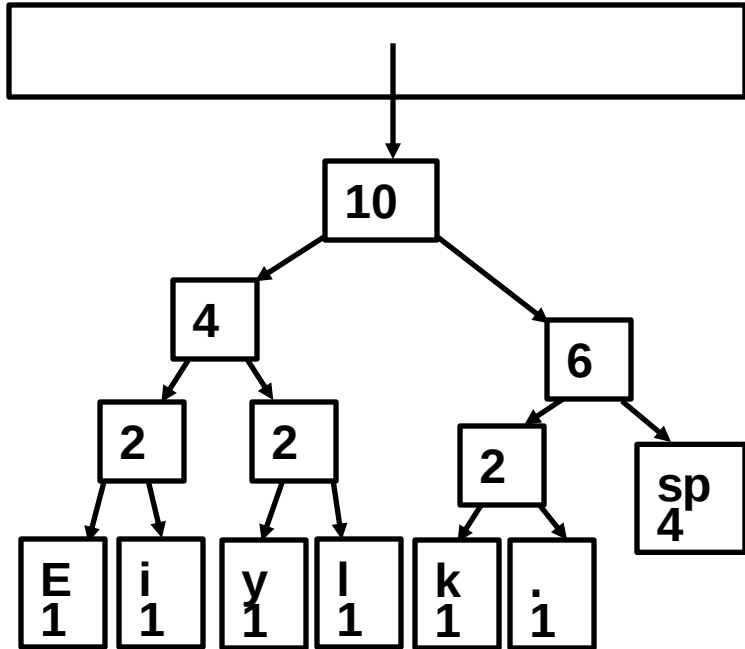
BUILDING A TREE



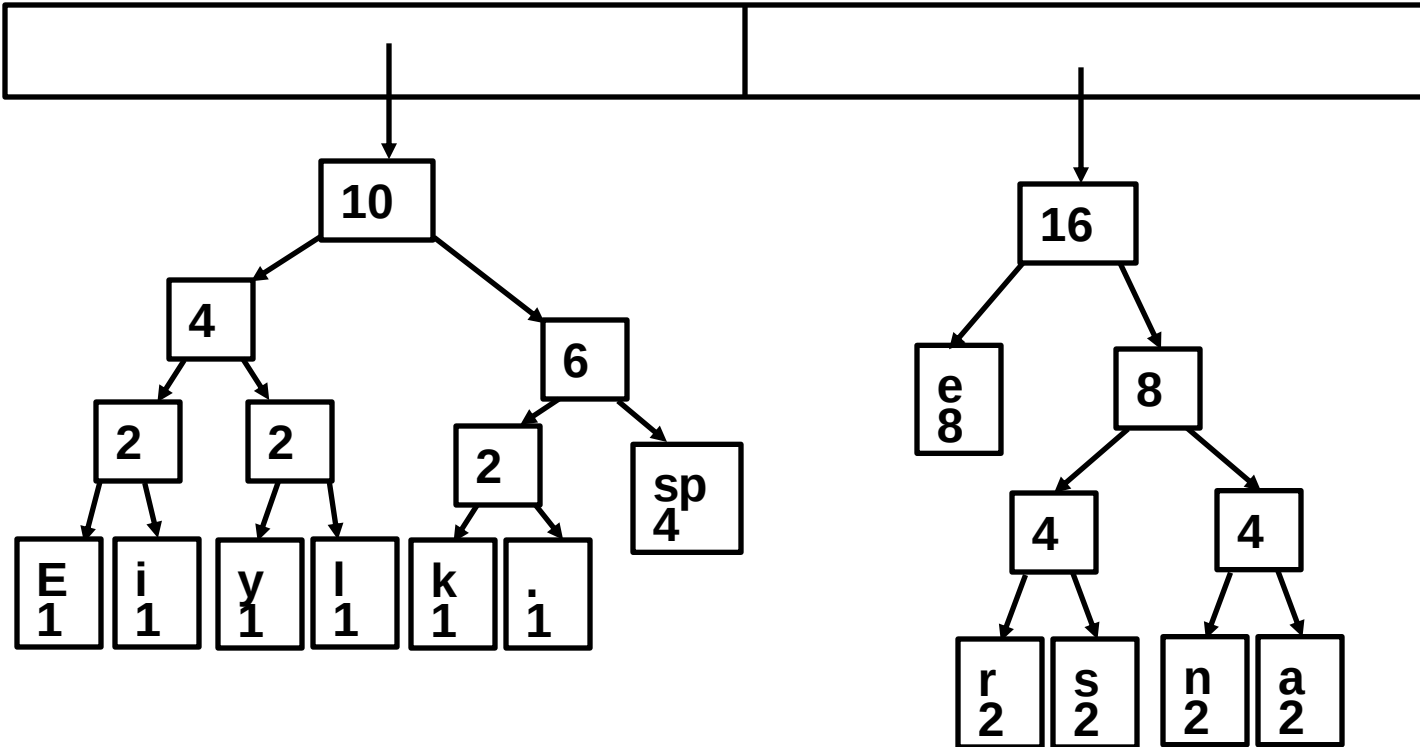
BUILDING A TREE



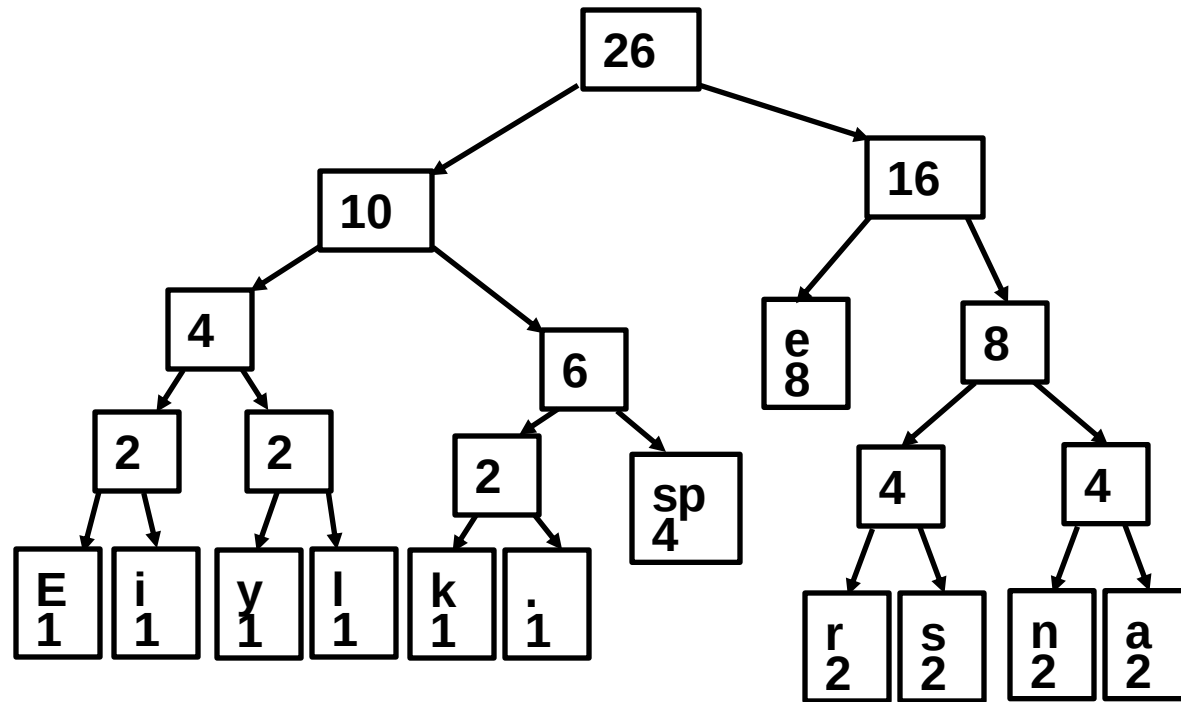
BUILDING A TREE



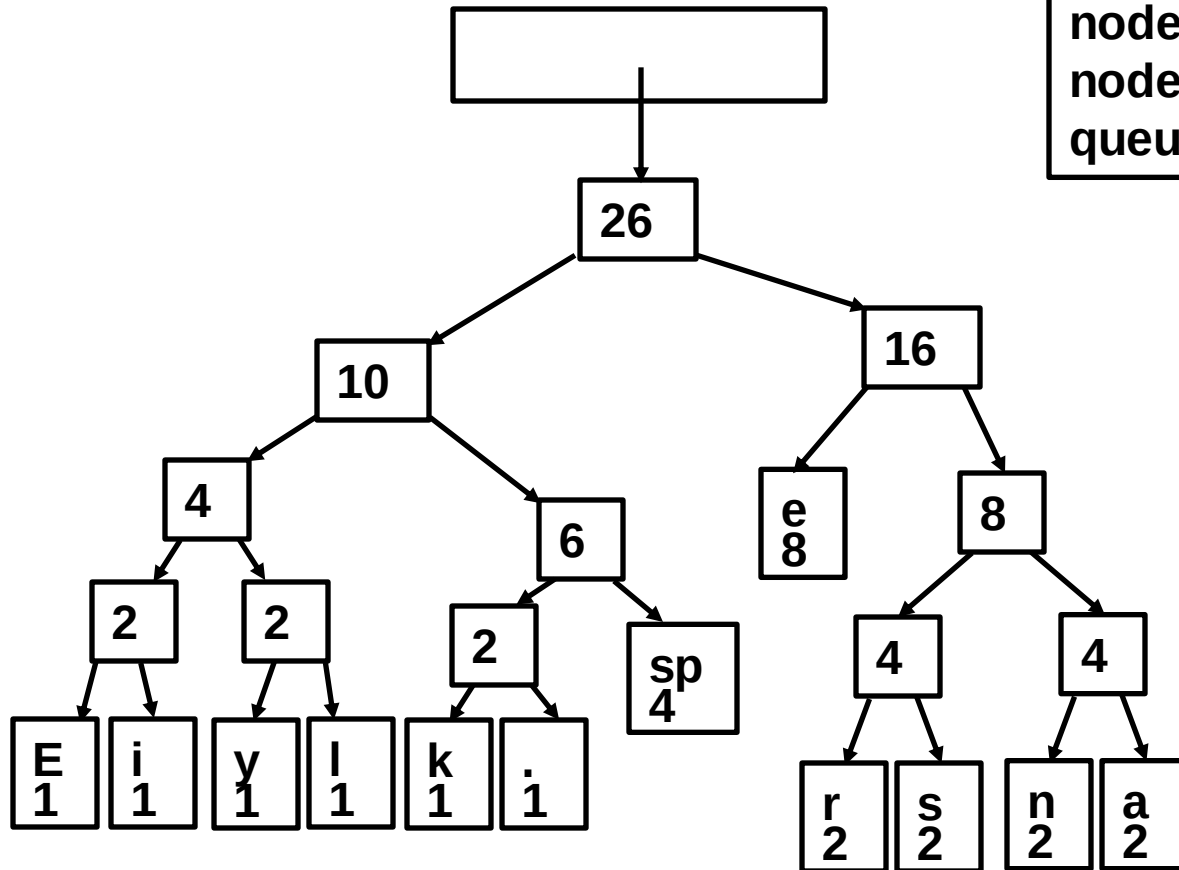
BUILDING A TREE



BUILDING A TREE



BUILDING A TREE



After enqueueing this node there is only one node left in priority queue.

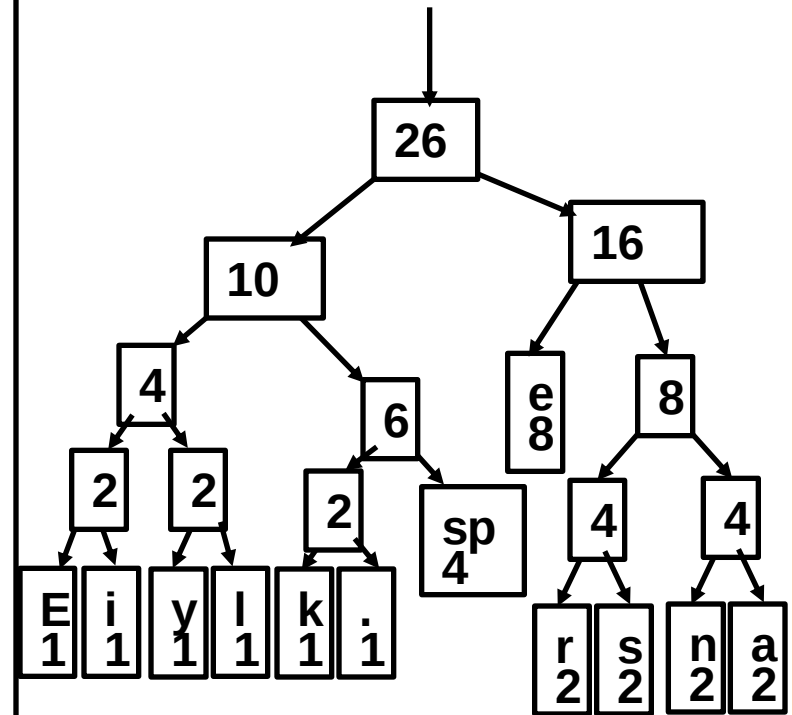


BUILDING A TREE

Dequeue the single node left in the queue.

This tree contains the new code words for each character.

Frequency of root node should equal number of characters in text.



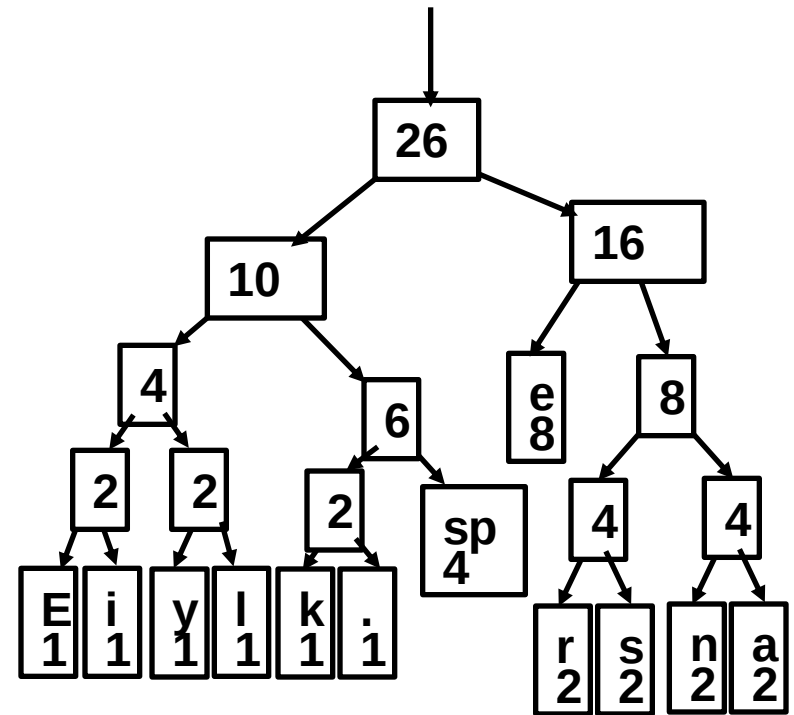
Eerie eyes seen near lake.

□ 26 characters

ENCODING THE FILE

TRAVERSE TREE FOR CODES

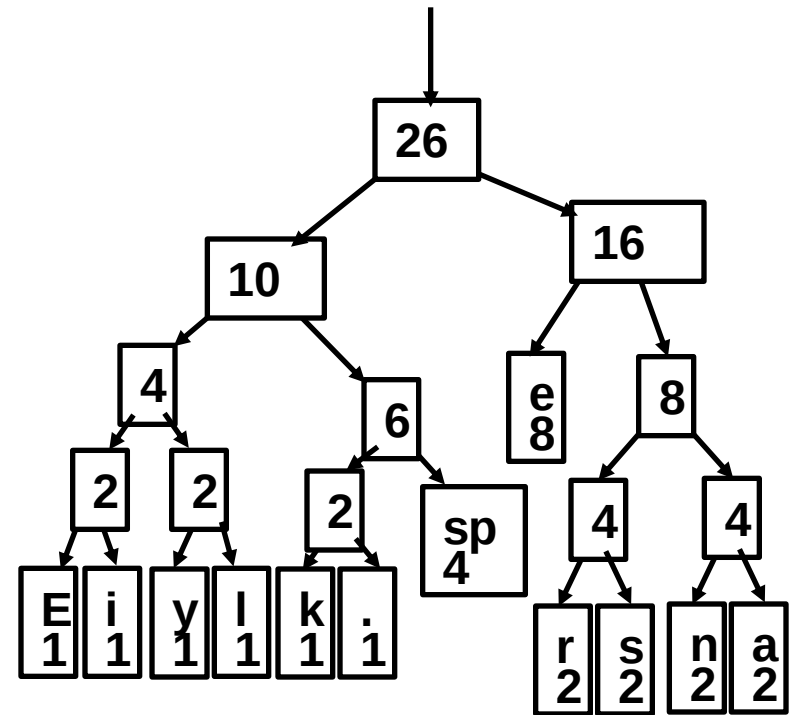
- Perform a traversal of the tree to obtain new code words
- Going left is a 0 going right is a 1
- code word is only completed when a leaf node is reached



ENCODING THE FILE

TRAVERSE TREE FOR CODES

Char	Code
E	0000
i	0001
y	0010
l	0011
k	0100
.	0101
space	011
e	10
r	1100
s	1101
n	1110
a	1111



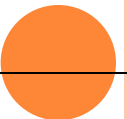
ENCODING THE FILE

- Rescan text and encode file using new code words

Eerie eyes seen near lake.

```
0000101100000110011100010101101101
00111110101111110001100111111010010
0101
```

Char	Code
E	0000
i	0001
y	0010
l	0011
k	0100
.	0101
space	011
e	10
r	1100
s	1101
n	1110
a	1111



ENCODING THE FILE RESULTS

- Have we made things any better?
- 73 bits to encode the text
- ASCII would take $8 * 26 = 208$ bits

```
00001011000001100111000101011011010011111010111110001100111111010  
0100101
```

- If modified code used 4 bits per character are needed.

Total bits

$4 * 26 = 104$. Savings not as great.



بعض من عيوب طريقة HUFFMAN :

- (1) لا بد ان يكون لدى كل من الطرفين (الطرف الذي يقوم بضغط الملف و الطرف الذي يقوم بفك الضغط) معلومات عن ال Huffman Tree و هذه تعتبر overhead على حجم الملف المضغوط .
- (2) لابد من قراءة الملف المراد ضغطه مرتين , مرة لإيجاد تكرار كل حرف داخل النص , و المرة الثانية عند إنشاء الملف المضغوط بعد توليد الأكواد للحروف , و في حالة الملفات الكبيرة الحجم هذه تعتبر مشكلة لأن العملية تستغرق وقت طويل .
- (3) في تطبيق اخر لطريقة Huffman مثل إرسال البيانات عبر الشبكات , نجد أن الطرف المستقبل لابد أن ينظر حتى ينتهي المرسل من عمله كاملا ثم يرسل له معلومات الشجرة الثنائية و البيانات المضغوطة , و هذا إهدار للوقت و المصادر Resource لأن احد الطرفين يظل بلا عمل (Idle) حتى ينتهي الآخر من عمله .

H.W

After a test men are honored or disgraced

قُلْ إِنَّ صَلَاتِي وَنُسُكِي وَمَحْيَايَ
وَمَمَاتِي لِلَّهِ رَبِّ الْعَالَمِينَ

